

CHƯƠNG 16: RA QUYẾT ĐỊNH PHỨC TẠP

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 16

- ◇ 16.1 Các Bài toán Quyết định Tuần tự (Sequential Decision Problems)
- ◇ 16.2 Thuật toán cho các MDP (Algorithms for MDPs)
- ◇ 16.3 Các Bài Toán Máy Đánh Bạc (Bandit Problems)
- ◇ 16.4 Các bài toán MDP có thể Quan sát Một phần (POMDP)
- ◇ 16.5 Các thuật toán giải POMDP

16.1 Bài toán Quyết định Tuần tự

◇ Trong các chương trước, tác nhân chỉ cần ra quyết định một lần (one-shot decision).

◇ **Bài toán Quyết định Tuần tự (Sequential Decision Problem):**

- Tác nhân phải đưa ra một *chuỗi* các quyết định.
- Sự lựa chọn ở hiện tại không chỉ đem lại phần thưởng ngay lập tức mà còn làm thay đổi trạng thái môi trường và kết quả của các quyết định trong tương lai.

◇ **Ví dụ:** Chơi cờ, lái xe, hoặc đầu tư tài chính.

16.1 Giới thiệu Quá trình Ra quyết định Markov

◇ Để mô hình hóa toán học các bài toán ra quyết định tuần tự trong môi trường không chắc chắn, ta sử dụng **Quá trình Ra quyết định Markov (Markov Decision Process - MDP)**.

◇ **Giả định Markov (Markov Assumption):** Xác suất chuyển sang trạng thái tiếp theo chỉ phụ thuộc vào trạng thái hiện tại và hành động hiện tại, không phụ thuộc vào lịch sử trước đó.

◇ **Mục tiêu của MDP:** Không phải là tìm một *chuỗi hành động* cố định (vì môi trường có yếu tố ngẫu nhiên), mà là tìm một **Chính sách (Policy - π)**.

- Chính sách $\pi(s)$ là một hàm ánh xạ từ một trạng thái s sang một hành động a tối ưu nhất.

16.1 Định nghĩa cấu trúc của một MDP

◇ Một MDP được định nghĩa chính thức bởi cụm 4 thành tố (S, A, P, R) :

1. S (**States**): Tập hợp tất cả các trạng thái có thể có của môi trường.
2. A (**Actions**): Tập hợp các hành động mà tác nhân có thể thực hiện.
3. $P(s'|s, a)$ (**Transition Model**): Xác suất chuyển từ trạng thái s sang trạng thái s' nếu thực hiện hành động a .
4. $R(s)$ (**Reward Function**): Phần thưởng thu được khi đạt tới trạng thái s (có thể âm nếu là hình phạt).

16.1 Hữu dụng theo thời gian & Hệ số chiết khấu

◇ Làm sao để so sánh lợi ích của một chuỗi phần thưởng kéo dài vô hạn?

- Chuỗi 1: $[1, 1, 1, 1, \dots] \rightarrow \infty$
- Chuỗi 2: $[2, 2, 2, 2, \dots] \rightarrow \infty$
- Cả hai đều tiến tới vô cùng nếu cộng dồn.

◇ **Hệ số chiết khấu (Discount factor - γ):** Một hằng số $0 \leq \gamma < 1$ dùng để giảm giá trị của phần thưởng nhận được trong tương lai.

- Hữu dụng của một chuỗi trạng thái $[s_0, s_1, s_2, \dots]$ là:
- $U([s_0, s_1, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots = \sum_{t=0}^{\infty} \gamma^t R(s_t)$
- Nhờ $\gamma < 1$, tổng này hội tụ về một số hữu hạn.

16.2 Thuật toán cho các MDP: Phương trình Bellman

- ◇ Để tìm được chính sách tối ưu π^* , ta cần xác định giá trị hữu dụng thực sự $U(s)$ của từng trạng thái.
- ◇ $U(s)$ không chỉ là phần thưởng nhận được ngay tại s , mà còn bao gồm **kỳ vọng tổng phần thưởng tương lai** nếu xuất phát từ s và đi theo chính sách tối ưu.
- ◇ Mỗi liên hệ này được biểu diễn bằng một công thức vĩ đại trong khoa học máy tính và kinh tế học: **Phương trình Bellman (Bellman Equation)**.

16.2 Chi tiết Phương trình Bellman

◇ **Phương trình Bellman:**

- $U(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} P(s'|s, a) U(s')$

◇ **Ý nghĩa:**

- $R(s)$: Lợi ích ngay lập tức.
- $\max_a$: Tác nhân chọn hành động tốt nhất.
- $\sum_{s'} P(s'|s, a) U(s')$: Tính kỳ vọng lợi ích tương lai tại các trạng thái tiếp theo s' , phụ thuộc vào xác suất chuyển đổi.

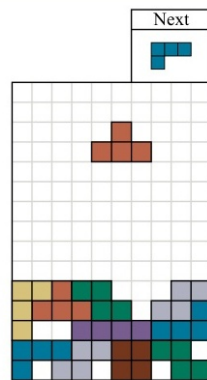
◇ Nếu có n trạng thái, ta sẽ có hệ n phương trình phi tuyến tính (vì có phép max).

16.2 Thuật toán Lặp Giá trị (Value Iteration)

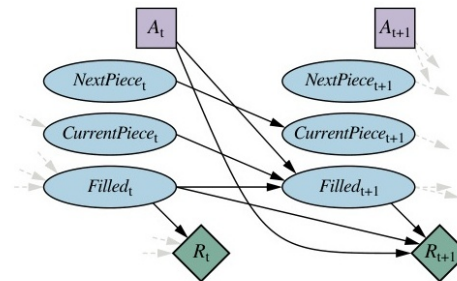
◇ Không thể giải hệ phương trình Bellman bằng phương pháp đại số thông thường. Ta dùng phương pháp lặp.

◇ **Thuật toán Lặp giá trị (Value Iteration):**

1. Khởi tạo $U_0(s) = 0$ cho mọi trạng thái.
2. Cập nhật hữu dụng của mọi s ở bước $i + 1$ dựa trên giá trị ở bước i :
 - $U_{i+1}(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} P(s'|s, a) U_i(s')$
3. Lặp lại bước 2 cho đến khi sự thay đổi giữa U_{i+1} và U_i rất nhỏ (đạt hội tụ).



(a)



(b)

16.2 Thuật toán Lặp Chính sách (Policy Iteration)

◇ Thuật toán Lặp giá trị có thể mất nhiều bước mới hội tụ chính xác.

◇ **Lặp Chính sách (Policy Iteration)** thay vì hội tụ giá trị $U(s)$, nó hội tụ trực tiếp trên không gian chính sách.

1. Khởi tạo ngẫu nhiên một chính sách π_0 .
2. **Đánh giá (Policy Evaluation):** Tính toán hàm $U(s)$ nếu chỉ đi theo đúng chính sách π_i . (Giải hệ phương trình tuyến tính, không có max).
3. **Cải tiến (Policy Improvement):** Cập nhật π_{i+1} bằng cách chọn các hành động tốt nhất dựa trên $U(s)$ vừa tính.
4. Lặp lại đến khi chính sách không còn thay đổi.

16.3 Các Bài Toán Máy Đánh Bạc (Bandit Problems)

- ◇ Một lớp đặc biệt của bài toán ra quyết định: Tác nhân đứng trước nhiều máy đánh bạc (Multi-armed bandits), mỗi máy có một xác suất trả thưởng vô danh khác nhau.
- ◇ Tác nhân phải ra quyết định chọn kéo cần gạt nào ở mỗi bước thời gian.
- ◇ **Sự cân bằng cốt lõi:** Khám phá (Exploration) vs. Khai thác (Exploitation).

16.3 Khám phá vs Khai thác & Chỉ số Gittins

◇ **Khám phá (Exploration):** Kéo thử một máy mới để tìm hiểu xác suất của nó (Chịu rủi ro ngắn hạn để thu lợi dài hạn).

◇ **Khai thác (Exploitation):** Liên tục kéo chiếc máy đang trả thưởng cao nhất hiện tại.

◇ **Chỉ số Gittins (Gittins Index):**

- Giải pháp toán học tối ưu cho bài toán Bandit độc lập.
- Nó gán một chỉ số cho mỗi trạng thái niềm tin của một máy, đánh giá tổng hợp cả giá trị khai thác hiện tại và tiềm năng khám phá tương lai.
- Thuật toán tối ưu: Tại mỗi bước, chọn máy có chỉ số Gittins cao nhất.

16.4 Bài toán có thể Quan sát Một phần (POMDP)

◇ **Hạn chế của MDP:** MDP giả định tác nhân luôn biết chính xác nó đang ở trạng thái s nào. Trong thực tế, cảm biến thường bị nhiễu hoặc điếm mù.

◇ **POMDP (Partially Observable Markov Decision Process):** Quá trình ra quyết định Markov quan sát một phần.

- Thêm vào mô hình cảm biến (Observation model): $O(o|s, a)$ - Xác suất nhận được quan sát o khi đang ở trạng thái s sau hành động a .

16.4 Trạng thái Niềm tin trong POMDP

◇ Vì tác nhân không biết s , nó phải duy trì một **Trạng thái Niềm tin (Belief State - b)**.

- $b(s)$ là xác suất mà tác nhân tin rằng nó đang ở trạng thái s .

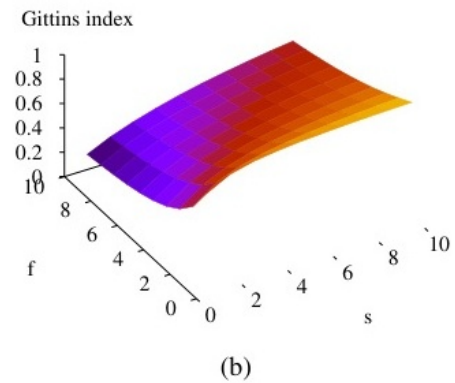
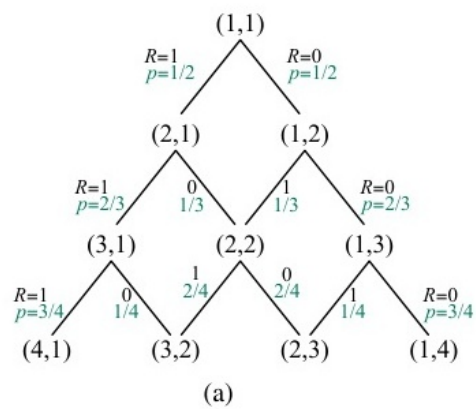
◇ Sau mỗi hành động a và quan sát o , trạng thái niềm tin được cập nhật theo định lý Bayes:

- $b'(s') = \alpha O(o|s', a) \sum_s P(s'|s, a) b(s)$
- α là hằng số chuẩn hóa.

◇ Một POMDP thực chất là một MDP mà ở đó không gian trạng thái chính là không gian liên tục của các Belief States.

16.5 Các thuật toán giải POMDP

- ◇ Mọi thuật toán của MDP (như Lặp giá trị) đều có thể áp dụng cho POMDP trên lý thuyết.
- ◇ Tuy nhiên, không gian trạng thái của MDP là hữu hạn (ví dụ: N trạng thái), trong khi không gian trạng thái niềm tin của POMDP là liên tục đa chiều (một điểm trong đơn hình $(N - 1)$ chiều).
- ◇ **Biểu diễn Hữu dụng POMDP:** Hàm $U(b)$ trong Lặp giá trị có thể được biểu diễn dưới dạng tập hợp các siêu phẳng (Hyperplanes) tuyến tính từng phần và lồi (Piecewise-linear and convex).



16.5 Sự bùng nổ không gian trong POMDP

◇ Mặc dù có các thuật toán giải chính xác POMDP, nhưng độ phức tạp tính toán cực kỳ khủng khiếp:

- Giải POMDP chính xác thuộc lớp PSPACE-hard.
- Số lượng siêu phẳng đại diện cho hàm Hữu dụng có thể tăng theo hàm mũ kép sau mỗi lần lặp.

◇ **Giải pháp thực tế:**

- Sử dụng thuật toán xấp xỉ trực tuyến (Online planning).
- Kết hợp Monte Carlo Tree Search (MCTS) với Bộ lọc Hạt (Particle Filtering) để xấp xỉ tập Belief States thay vì tính toán toàn bộ không gian liên tục.

Tóm tắt Chương 16

- ◇ **MDP (Markov Decision Process)** cung cấp nền tảng chuẩn xác cho các bài toán ra quyết định chuỗi thời gian, nơi hành động hiện tại ảnh hưởng đến tương lai.
- ◇ **Phương trình Bellman** là cốt lõi để giải quyết MDP thông qua các thuật toán *Lập giá trị* và *Lập chính sách*.
- ◇ **Bài toán Bandit** định hình lại triết lý tối ưu giữa Khám phá cái mới và Khai thác cái tốt nhất hiện tại.
- ◇ **POMDP** phản ánh thực tế khắc nghiệt nhất: Môi trường vừa ngẫu nhiên, vừa bị che khuất. Trạng thái niềm tin (Belief State) là công cụ duy nhất để duy trì lý trí.